

UNIVERSIDAD NACIONAL DE SAN AGUSTÍN

FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



COMPUTACIÓN GRÁFICA, VISIÓN COMPUTACIONAL Y MULTIMEDIA

Grupo B

Docente: Rene Alonso Nieto Valencia

Estudiante: Jhoer Luis Vera Zapacayo

AREQUIPA – PERÚ
2026

Informe del desarrollo de la aplicación

Diseño del escenario (fondos, elementos decorativos, límites)

El escenario fue diseñado para ofrecer una experiencia visual en 2.5 en donde el escenario sería en 2D y los objetos serían en 3d. Se utilizaron imágenes de fondo espaciales en formato PNG. Los límites del escenario fueron establecidos con Colisionadores invisibles (GameObjects con BoxCollider2D) que impiden que el jugador o los enemigos salgan del área visible de juego. Esto asegura que la acción se mantenga contenida en la pantalla y evita errores de lógica por fuera del campo visual.

Herramientas de Unity utilizadas (Tilemap, SpriteRenderer, etc.)

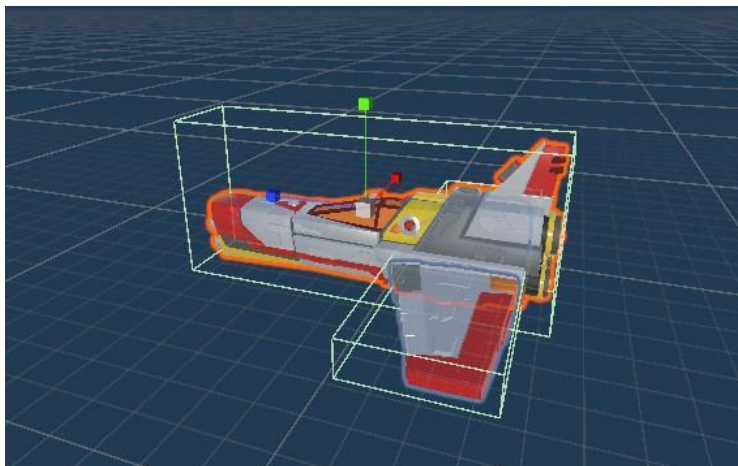
- BoxCollider2D / CircleCollider2D: Empleados para detectar colisiones en el escenario.
- Rigidbody2D: Asociado a objetos que se mueven (jugador, enemigos, proyectiles) para aplicar físicas y movimiento controlado.
- Canvas y UI Text/Image: Utilizados para mostrar la interfaz de usuario (vidas, score, nivel actual).

Además, muchos de los modelos visuales utilizados fueron descargados desde la **Unity Asset Store**. Se eligieron paquetes gratuitos y optimizados lo que permitió acelerar el desarrollo y mantener una coherencia visual de calidad.

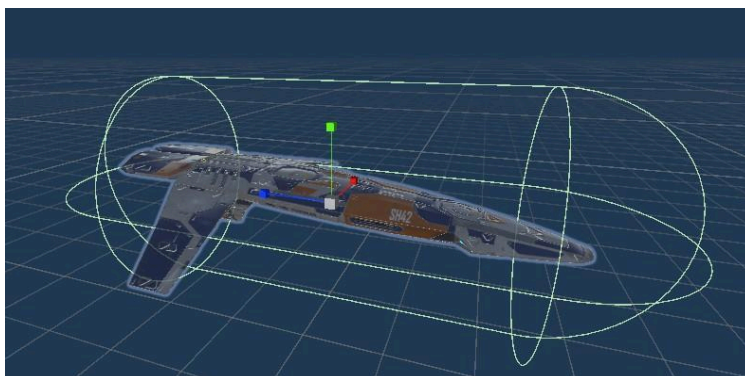
Estructura del proyecto (GameObjects, scripts, prefabs)

- GameObjects Principales:
 - Nave (jugador con control y disparo)
 - Enemigo (enemigos con AI)
 - misil (prefab de misiles)
 - UIManager (maneja textos e interfaz)
 - Game controller (control de niveles y lógica general)

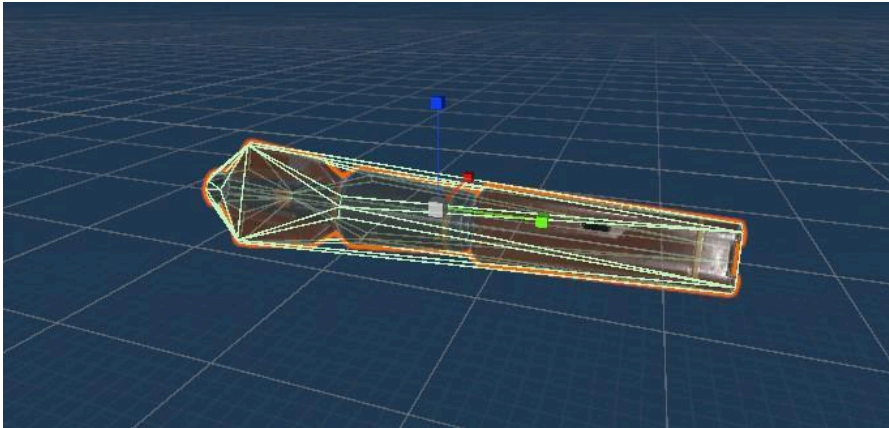
- Scripts Clave:
 - bala: controla el movimiento y disparo del jugador y del enemigo.
 - Colisiones: gestiona el comportamiento de cómo se dan las colisiones entre objetos ya sea los misiles y las naves .
 - controladores: controlar el movimiento del jugador y la mecánica de disparo.
 - Diseño_de_juego: estionar el puntaje del jugador y generar enemigos u obstáculos
 - mover:**mover proyectiles (como misiles o balas) en una dirección constante,**
 - nave_enemiga: dispare proyectiles automáticamente
- Prefabs:
 - nave



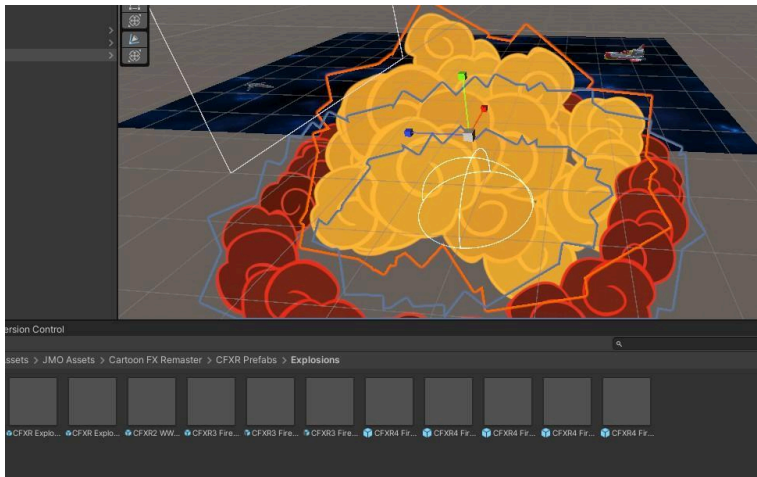
nave enemiga



misil



explosiones



Explicación de los script (las explicaciones se puso como comentarios en el código para poder ser más dinámico con esta misma)

bala.cs

```
CSharp
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

// Esta clase controla el comportamiento de una bala o proyectil en Unity
public class bala : MonoBehaviour
{
    // Velocidad a la que se moverá la bala
    public float velocidad;

    // Update se llama una vez por frame
    void Update()
    {
        // Mueve la bala hacia arriba (eje Y local)
```

```

        transform.Translate(Vector3.up * velocidad * Time.deltaTime);
    }
}

```

colisiones.cs

```

CSharp
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

// Maneja las colisiones entre objetos
public class colisiones : MonoBehaviour
{
    public GameObject explosion;
    public GameObject explosionjugador;
    public int scorevalue;

    private controlador gamecontroller;

    void Start()
    {
        gamecontroller = GameObject.FindWithTag("GameController")
            .GetComponent<controlador>();
    }

    private void OnTriggerEnter(Collider other)
    {
        if (other.CompareTag("Boundary"))
        {
            return;
        }

        Instantiate(explosion, transform.position, transform.rotation);

        if (other.CompareTag("nave"))
        {
            Instantiate(explosionjugador,
                other.transform.position,
                other.transform.rotation);
        }

        gamecontroller.addscore(scorevalue);

        Destroy(other.gameObject);
        Destroy(gameObject);
    }
}

```

controladores.cs

```
CSharp
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

// Control del jugador
public class controladores : MonoBehaviour
{
    private Rigidbody rig;

    public float velocidad = 5f;

    public float limiteXMin;
    public float limiteXMax;
    public float limiteZMin;
    public float limiteZMax;

    public GameObject shot;
    public Transform shotSpawn;

    public float fireRate = 0.5f;
    private float nextFire = 0f;

    void Update()
    {
        if (Input.GetButton("Fire1") && Time.time > nextFire)
        {
            nextFire = Time.time + fireRate;
            Instantiate(shot, shotSpawn.position, shotSpawn.rotation);
        }
    }

    void Awake()
    {
        rig = GetComponent<Rigidbody>();
    }

    void FixedUpdate()
    {
        float movehorizontal = Input.GetAxis("Horizontal");
        float movevertical = Input.GetAxis("Vertical");

        Vector3 movimiento = new Vector3(
            movehorizontal * velocidad,
            0f,
            movevertical * velocidad
        );
    }
}
```

```

    );

    rig.velocity = movimiento;

    Vector3 posicionLimite = rig.position;

    posicionLimite.x = Mathf.Clamp(posicionLimite.x, limiteXMin,
limiteXMax);
    posicionLimite.z = Mathf.Clamp(posicionLimite.z, limiteZMin,
limiteZMax);

    rig.position = posicionLimite;
}
}

```

diseño_juego.cs

```

CSharp
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;

// Controlador principal del juego
public class controlador : MonoBehaviour
{
    public GameObject hazard;
    public Vector3 spawnvalues;

    private int score;

    public Text scoretext;

    void Start()
    {
        score = 0;
        Updatescore();
        spawn();
    }

    void spawn()
    {
        Vector3 spawnposition = new Vector3(
            Random.Range(-spawnvalues.x, spawnvalues.x),
            spawnvalues.y,
            spawnvalues.z
        );
    }
}

```

```

        );

        Instantiate(hazard, spawnposition, Quaternion.identity);
    }

    public void addscore(int value)
    {
        score += value;
        Updatescore();
    }

    void Updatescore()
    {
        scoretext.text = "Score: " + score;
    }
}

```

mover.cs

```

CSharp
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

// Movimiento constante
public class mover : MonoBehaviour
{
    public float velocidad;

    void Update()
    {
        transform.Translate(Vector3.up * velocidad * Time.deltaTime);
    }
}

```

nave_enemiga.cs

```

CSharp
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

// Disparo automático del enemigo
public class nave_enemiga : MonoBehaviour
{

```



```

public GameObject shot;
public Transform shotspanw;

public float delay;
public float firerate;

void Start()
{
    InvokeRepeating("Fire", delay, firerate);
}

void Fire()
{
    Instantiate(shot, shotspanw.position, shotspanw.rotation);
}
}

```

Referencias

- [1] Goldstone, W. (2011). *Unity Game Development Essentials*. Packt Publishing Ltd.
- [2] Hocking, J. (2018). *Unity in Action: Multiplatform Game Development in C# with Unity 2018* (2nd ed.). Manning Publications.
- [3] Gregory, J. (2018). *Game Engine Architecture* (3rd ed.). CRC Press.
- [4] Griffiths, R., & Thorn, A. (2020). *Game Programming with Unity and C#: A Beginner's Guide*. Apress.
- [5] Adam Watkins (2011). *Introduction to Game Design, Prototyping, and Development*. New Riders.